UNISWAP V3 / V4 PAIRVAULT FEETO ARCHITECTURE

LPFY · SMART CONTRACTS

Smart contract architecture

Contract map for the markets stack: Uniswap V3/V4 LP NFT adapters, ERC-4626 PairVaults, MarketLendingModule, oracle, rates, liquidation swaps (including fee fallback), and how funds move — with repo references.

Overview · System map · Custody · Flows · Contracts · Fees · External · Repo layout · Deployed

---

1. Overview

Lenders supply **Circle USDC** into isolated per-pair **ERC-4626 PairVaults**. Borrowers lock a **Uniswap V3 or V4 LP NFT** and borrow USDC from the matching pair. Interest accrues on-chain. Underwater loans are liquidated by unwinding the Uniswap position through the V3/V4 adapter, swapping to USDC, and repaying the vault — callable by the market owner (or an optional automation receiver). After debt is pushed back to the vault, a protocol fee may go to `feeTo`.

Idle USDC lives in **PairVault**, not on the market. Legacy `lendUsdc` / `withdrawLender` revert (`UsePairVault`).

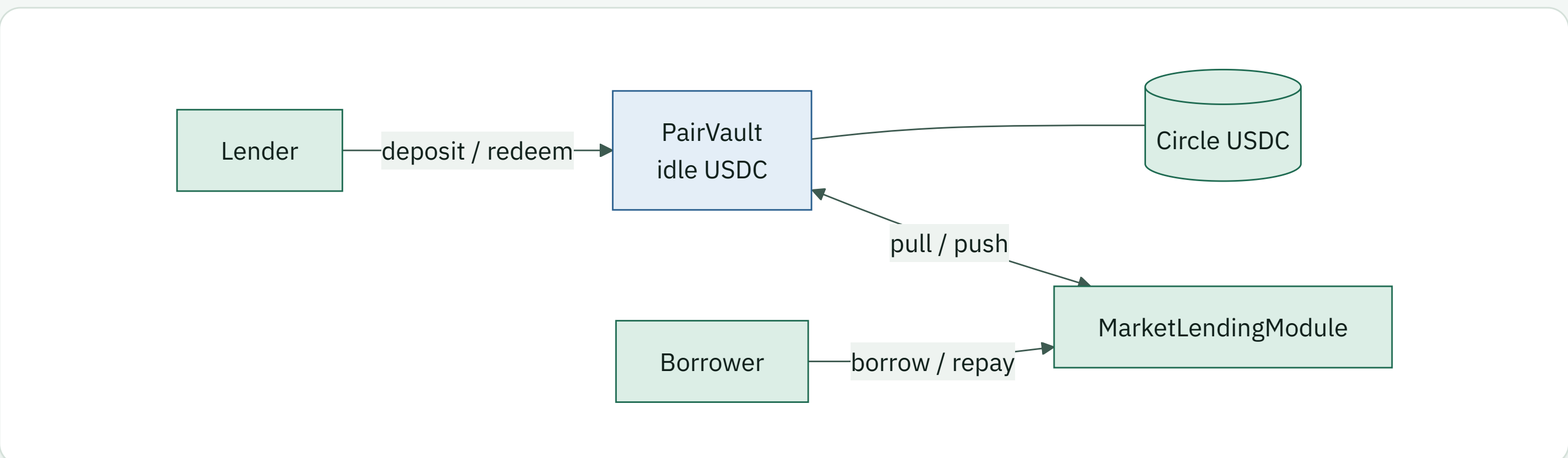
Uniswap surfaces used in this platform: V3 NonfungiblePositionManager + Factory/Pool (via `V3Adapter`); V4 PositionManager + StateView (via `V4Adapter`); V3 SwapRouter02 `exactInputSingle` for liquidation recovery (plus optional `FeeFallbackSwapRouter`). We integrate the **on-chain AMM (v3 + v4)** directly — not the Uniswap SDK, Trading API, CCA, or custom hooks.

Canonical address file: `smart-contracts/deployments/sepolia-markets.json`. Repo: github.com/unblocktechie/LPFy.

2. System map

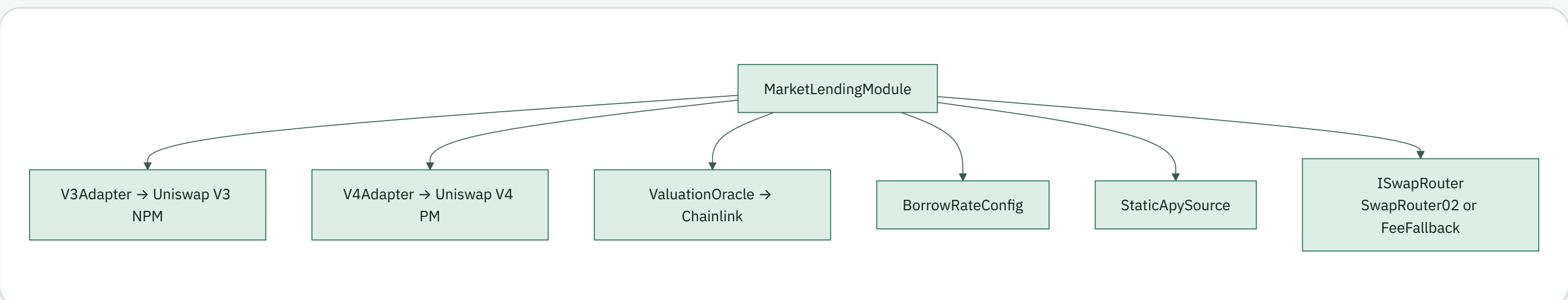
Three small diagrams instead of one tangled map — easier to see where USDC lives and who calls whom.

2.1 Money flow (lend / borrow)


Lender
deposit / redeem
PairVault
idle USDC
Circle USDC
pull / push
Borrower
borrow / repay
MarketLendingModule

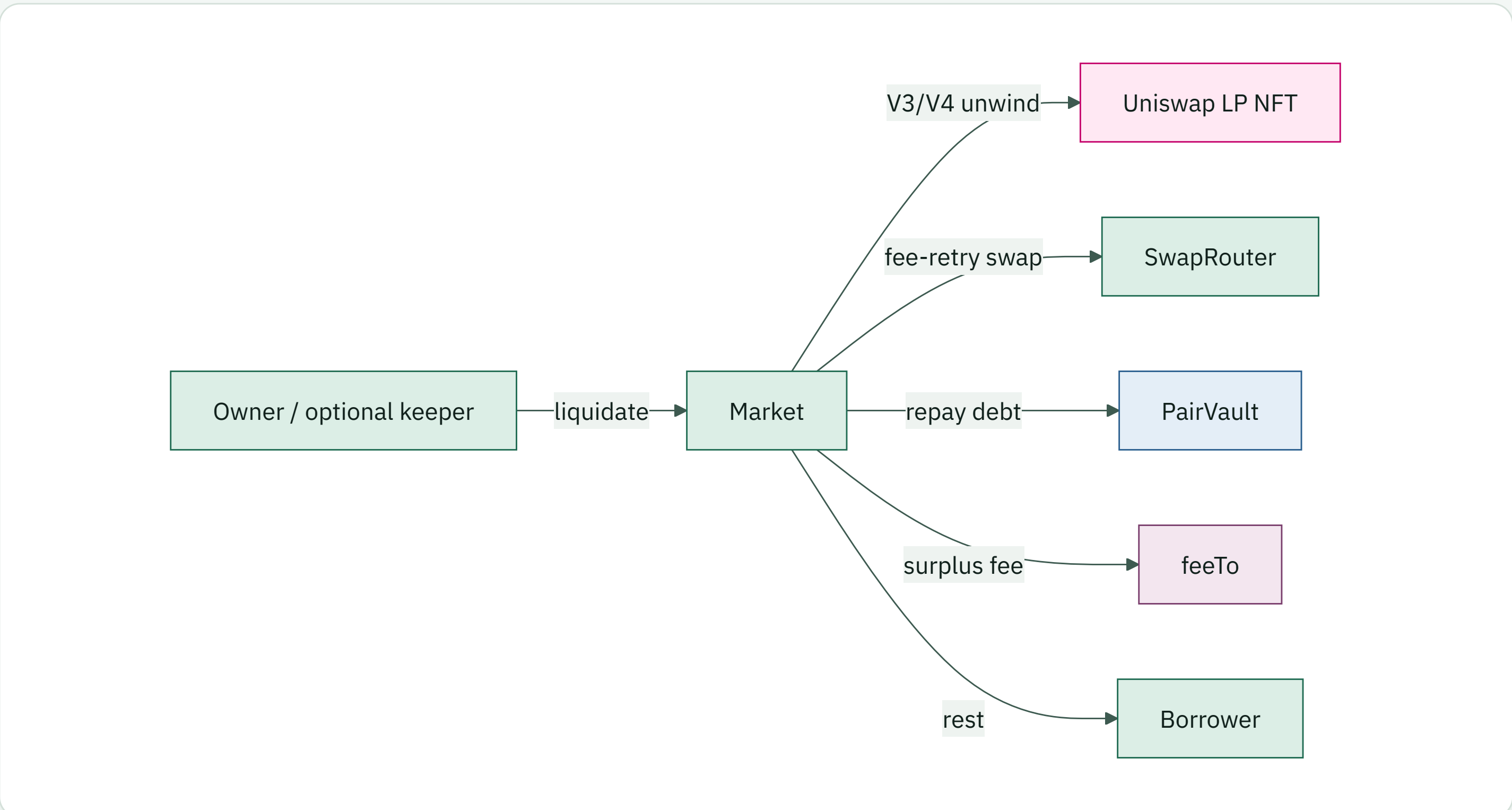
Lender ↔ PairVault only. USDC idle balance is in the vault. Market pulls to the borrower and pushes on repay — it does not hold idle lender cash.

2.2 Risk stack (what Market uses)


MarketLendingModule
V3Adapter → Uniswap V3 NPM
V4Adapter → Uniswap V4 PM
ValuationOracle → Chainlink
BorrowRateConfig
StaticApySource
ISwapRouter SwapRouter02 or FeeFallback

Adapters custody Uniswap LP NFTs. Oracle prices collateral. APR/APY are config. Router is liquidation-only (`setSwapRouter`).

2.3 Liquidation (unwind + fee)


Owner / optional keeper
liquidate
Market
V3/V4 unwind
Uniswap LP NFT
fee-retry swap
SwapRouter
repay debt
PairVault
surplus fee
feeTo
rest
Borrower

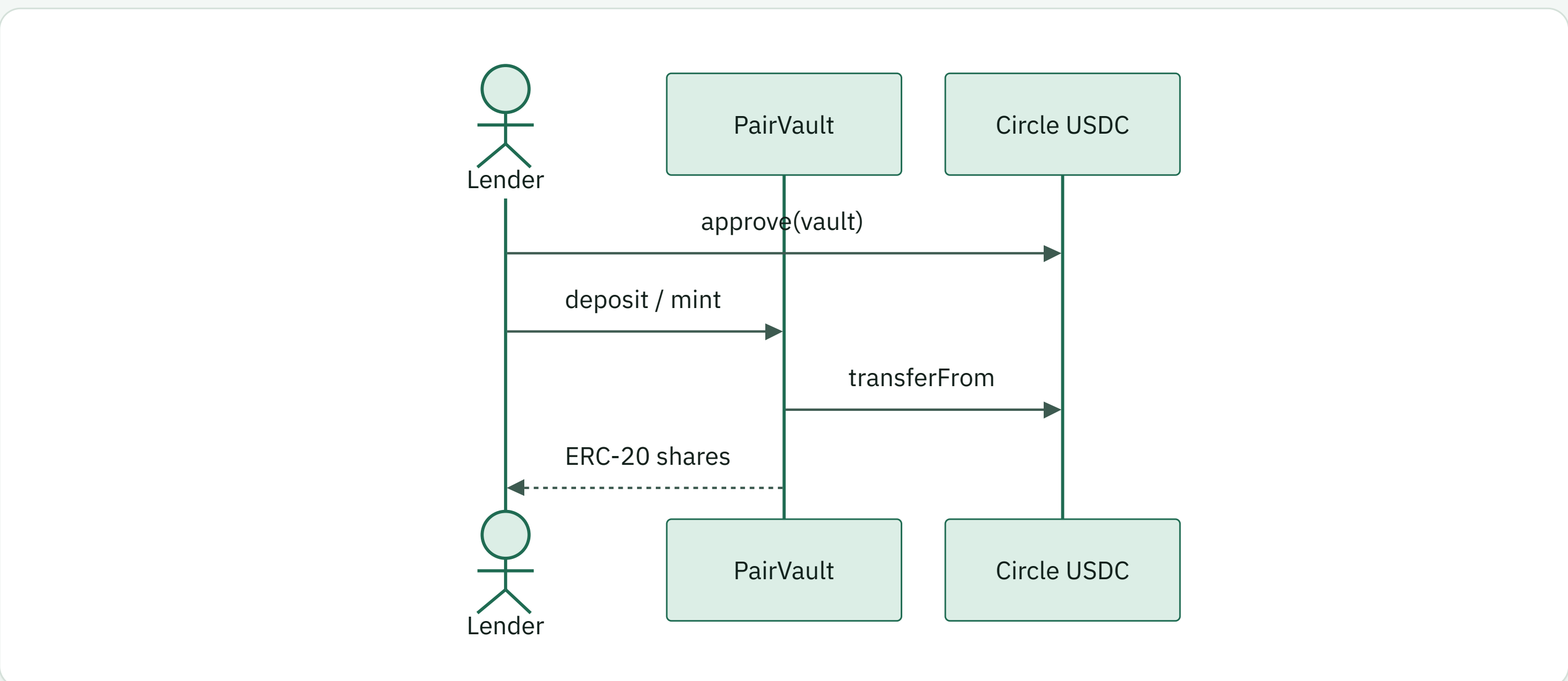
Authorized caller triggers liquidate. Adapter unwinds the Uniswap position; `_swapToUsdc` tries the LP fee then standard V3 tiers (V4 fees like 20 are not V3 pools). Debt returns to PairVault first; then feeTo; then borrower surplus. Optional CRE path: workflow → Forwarder → `CreLiquidationReceiver`.

3. Who holds what

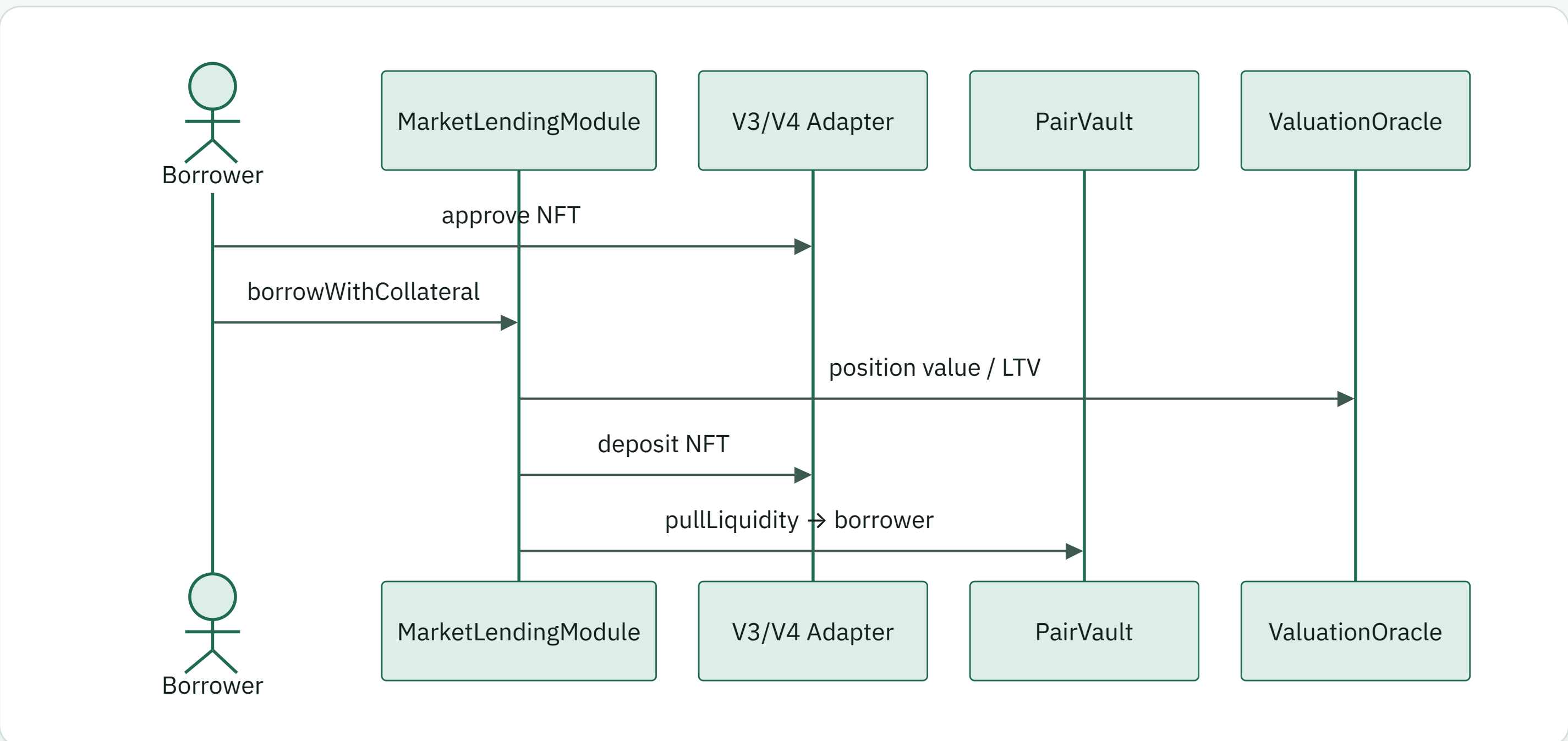
ASSET	CUSTODY	NOTES
Idle Circle USDC	`PairVault` (per pair)	Lender deposits / redeems here
Outstanding principal + interest	Tracked on market; owed to vault	Included in vault `totalAssets()`
DEX V3 LP NFT	`V3Adapter`	Unwind on liquidate
DEX V4 LP NFT	`V4Adapter`	Native ETH mapped to WETH for pair / valuation
Liquidation fee	Transferred to `feeTo`	Only from surplus after vault repaid

4. Core flows

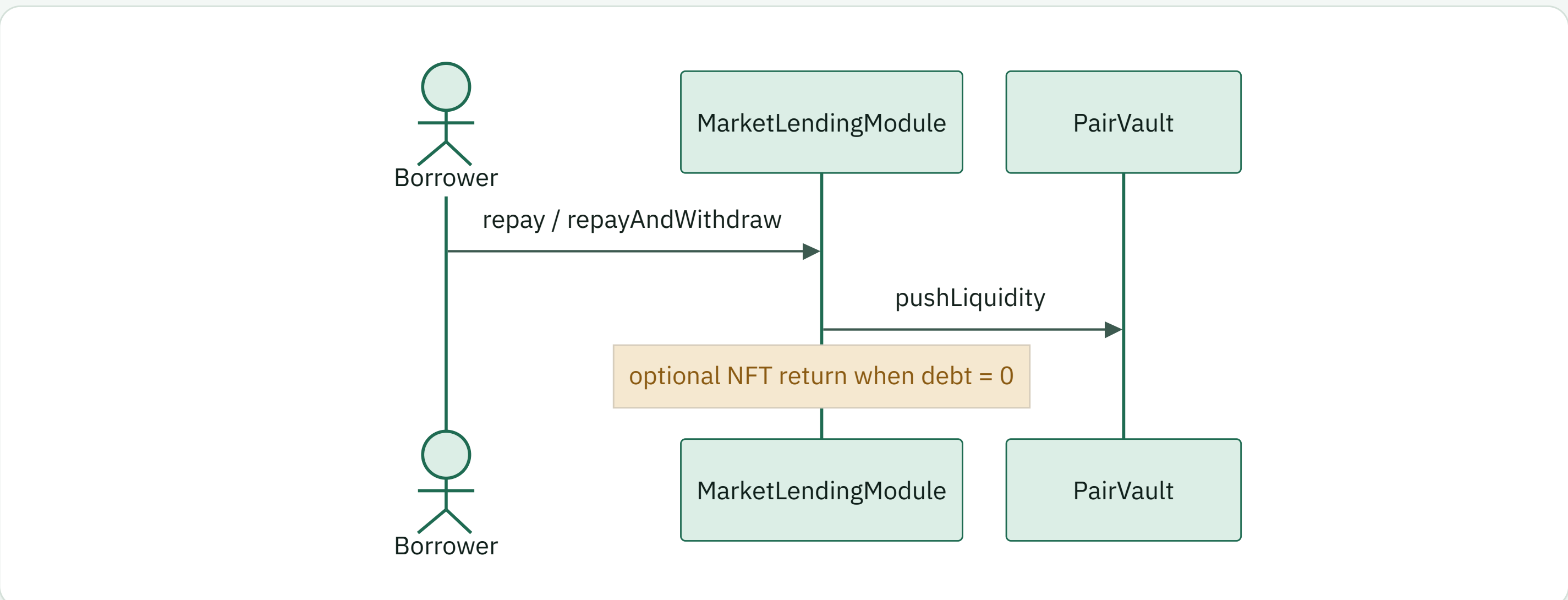
4.1 Supply (lender → vault)


Lender
PairVault
Circle USDC
approve(vault)
deposit / mint
transferFrom
ERC-20 shares
Lender
PairVault
Circle USDC

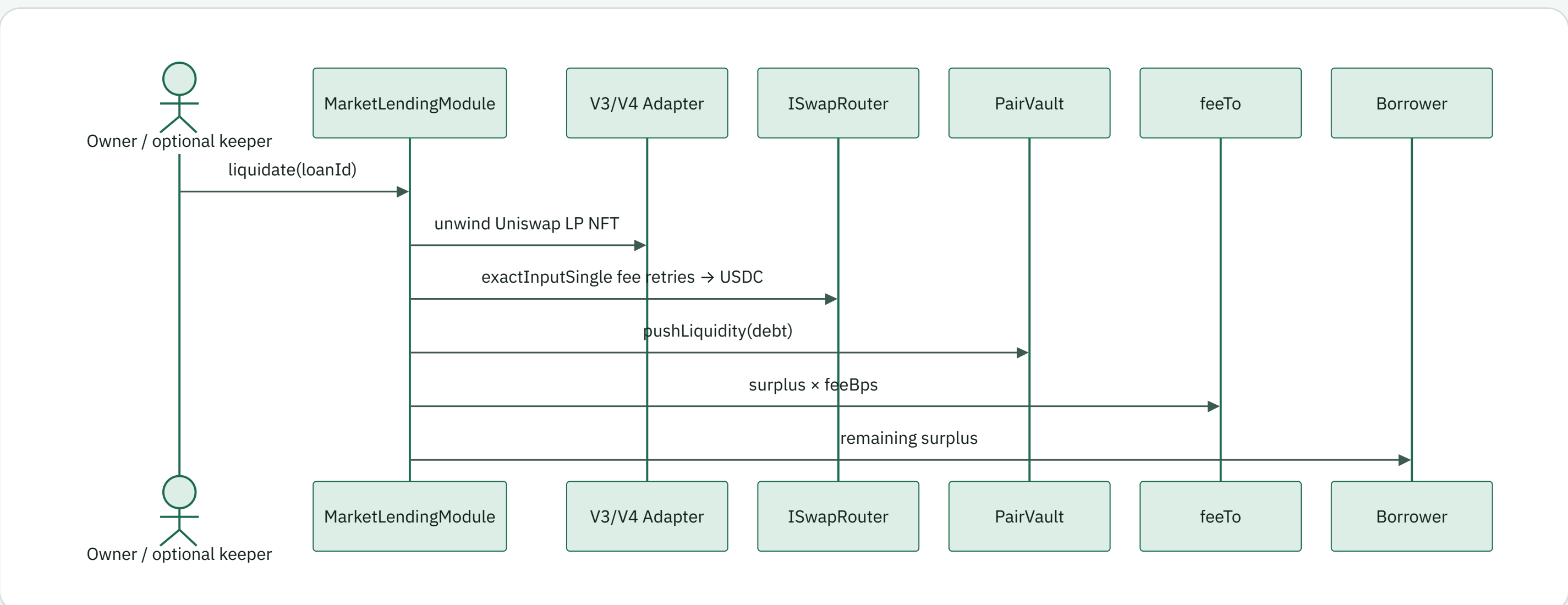
4.2 Borrow (market pulls vault)


Borrower
MarketLendingModule
V3/V4 Adapter
PairVault
ValuationOracle
approve NFT
borrowWithCollateral
position value / LTV
deposit NFT
pullLiquidity → borrower
Borrower
MarketLendingModule
V3/V4 Adapter
PairVault
ValuationOracle

4.3 Repay (market pushes vault)


Borrower
MarketLendingModule
PairVault
repay / repayAndWithdraw
pushLiquidity
optional NFT return when debt = 0
Borrower
MarketLendingModule
PairVault

4.4 Liquidate


Owner / optional keeper
MarketLendingModule
V3/V4 Adapter
ISwapRouter
PairVault
feeTo
Borrower
liquidate(loanId)
unwind Uniswap LP NFT
exactInputSingle fee retries → USDC
pushLiquidity(debt)
surplus × feeBps
remaining surplus
Owner / optional keeper
MarketLendingModule
V3/V4 Adapter
ISwapRouter
PairVault
feeTo
Borrower

`previewLiquidation` is public. Execute requires owner or an authorized automation receiver. Swaps use the configured router (`SwapRouter02` and/or `FeeFallbackSwapRouter`).

5. Contracts — purpose & surfaces

ERC-4626

PairVault

`smart-contracts/contracts/markets/PairVault.sol`

Per-pair idle USDC vault. Standard ERC-4626 deposit / mint / withdraw / redeem. Only the linked market may `pullLiquidity` / `pushLiquidity`. `totalAssets` = idle + `market.assetsOwedToVault(pairId)`.

`deposit / mint` `withdraw / redeem` `pullLiquidity` `pushLiquidity` `idleAssets` `totalAssets` `maxWithdraw / maxRedeem`

CORE

MarketLendingModule

`smart-contracts/contracts/markets/MarketLendingModule.sol`

Loans, LTV, interest accrual, Uniswap LP NFT liquidation (adapter unwind + fee-retry USDC swaps), vault registry (`vaultOf`), and fee config (`feeTo` , `liquidationFeeBps`). Does not hold idle lender cash.

Borrower

`previewBorrow` `borrowWithCollateral` `borrowMore` `repay` `repayAndWithdraw` `withdrawCollateral` `currentDebt`

Vault accounting

`vaultOf` `setVault` `assetsOwedToVault` `availableToBorrow`

Liquidation / health

`isLiquidatable` `previewLiquidation` `findFirstLiquidatableLoan` `liquidate` `setSwapRouter` `accrueInterest`

`_swapToUsdc` tries fees `[LP fee, 500, 3000, 10000, 100]` so V4 pool fees (e.g. 20) still liquidate via a V3 router pool when one exists.

RATES

BorrowRateConfig

`contracts/markets/BorrowRateConfig.sol`

Per-pair borrow APR and whether a pair is enabled.

`setPair` `getBorrowAprBps` `isPairSupported` `pairIdOf`

RATES

StaticApySource

`contracts/markets/StaticApySource.sol`

Display lender APY only — does not move funds.

`setApyBps` `setDefaultApyBps` `getLenderApyBps`

ORACLE

ValuationOracle

`contracts/ValuationOracle.sol`

LP amounts × Chainlink USD → debt-asset units for LTV.

`getPositionValueUsd` `getTokenPriceUsd` `convertUsdToDebtAsset` `setFeed` `setAdapter`

CUSTODY

V3Adapter

`contracts/adapters/V3Adapter.sol`

Holds V3 LP NFTs; deposit / withdraw / unwind for liquidation.

`deposit` `withdraw` `unwind` `owns` `getPositionMeta` `getAmountsForValuation`

CUSTODY

V4Adapter

`contracts/adapters/V4Adapter.sol`

Same role for Uniswap V4 positions; ETH → WETH mapping for pair identity. Unwind collects tokens; market still swaps via the V3-style `ISwapRouter`.

`deposit` `withdraw` `unwind` `registerPosition` `getAmountsForValuation`

LIQUIDATION

FeeFallbackSwapRouter

`contracts/markets/FeeFallbackSwapRouter.sol`

Drop-in `exactInputSingle` wrapper around Uniswap SwapRouter02. Retries fee tiers for already-deployed markets via `setSwapRouter` (no market upgrade). Newer market builds also retry fees inside `_swapToUsdc`.

`exactInputSingle` `inner (SwapRouter02)`

OPTIONAL

CreLiquidationReceiver

`contracts/markets/CreLiquidationReceiver.sol`

Optional automation: Forwarder `onReport` → `market.liquidate`. Does not configure fees or define the Uniswap collateral model.

`needsUpkeep` `onReport` `setForwarder` `setMarket`

SUPPORT

Libraries & interfaces

AREA	ROLE
`IPairVault` / vault accounting interfaces	Pull/push + assets owed surface
`IPositionAdapter`	Adapter surface for market + oracle
`IValuationOracle`	Oracle surface
`TickMath` / `FullMath` / `LiquidityAmounts` / `PoolId`	DEX math for valuation
`LendingTypes`	Shared protocol version (V3 / V4)
`mocks/*`	Hardhat tests only

6. Fee structure (on market)

KNOB	MEANING
`liquidationFeeBps`	% of surplus after vault debt repaid (`1000` = 10%)
`feeTo`	Fee recipient — independent of Ownable `owner`

Order on liquidate: repay PairVault → fee to `feeTo` → remainder to borrower. Shortfall → no fee. Set via `setLiquidationFeeBps` / `setFeeTo` or redeploy env vars.

7. External dependencies

DEPENDENCY	USED FOR	CALLED BY
ERC20 Circle USDC	Debt asset & vault asset	PairVault, MarketLendingModule
DEX V3 NonfungiblePositionManager	Transfer / decrease / collect LP NFTs	V3Adapter
DEX V4 PositionManager + StateView	V4 custody & unwind	V4Adapter
DEX SwapRouter02	Non-USDC → USDC on liquidation (V3 pools). May be wrapped by `FeeFallbackSwapRouter`.	MarketLendingModule via `setSwapRouter`
CHAINLINK price feeds	Token USD for LTV	ValuationOracle
CRE KeystoneForwarder (optional)	Deliver signed reports for auto-liquidate	CreLiquidationReceiver

8. Repo layout (contracts package)

```
smart-contracts/
├── contracts/
│   ├── markets/
│   │   ├── MarketLendingModule.sol
│   │   ├── PairVault.sol
│   │   ├── FeeFallbackSwapRouter.sol
│   │   ├── CreLiquidationReceiver.sol
│   │   ├── BorrowRateConfig.sol
│   │   └── StaticApySource.sol
│   ├── adapters/          # V3Adapter, V4Adapter
│   ├── ValuationOracle.sol
│   ├── interfaces/
│   ├── libraries/
│   └── mocks/             # tests only
├── scripts/               # redeploy, fee-router, fee setters, local keeper
├── test/
└── deployments/           # sepolia-markets.json
```

GIT

Repository

github.com/unblocktechie/LPFy

Key packages: `smart-contracts/` · `web/` · `cre/` · `docs/`

Deployed contracts (Sepolia)

From `smart-contracts/deployments/sepolia-markets.json`. External Uniswap / Chainlink / Circle addresses are in section 7.

CONTRACT	ADDRESS
`MarketLendingModule`	`0x618a17C71b1D3922172E1b77D6e3D17704DE9f41`
`ValuationOracle`	`0x16E03515cf524B5D0D0db6E44F342ef0af7301e3`
`V3Adapter` / `V4Adapter`	`0x48ee590E073f5ae2FDFCEA465e6281115308dDc9` / `0x646DB0572422424947262fC3Bda2940f51504759`
`FeeFallbackSwapRouter` set as market swapRouter (wraps SwapRouter02)	`0xEB66D88296C1694d79D578A700cb4EE0d6dC8baf`
`BorrowRateConfig`	`0x96ff2ce7B8AEfEadCAF460f41af613e7640AE20B`
`StaticApySource`	`0x02874283bd490350Ace341bc9Cdd4E7A686CB4E1`
`CreLiquidationReceiver`	`0x32866b81408d0260259BC0109b9F0a300Cdca132`
`PairVault` USDC/WETH	`0xE47ABe4b88Fa2F47f4913710e8ecfBDE1205A5dc`
`PairVault` USDC/USDT	`0x9cf1CF796147DD6c4Ff2f57E91F67fe14397d8eF`
`PairVault` USDC/WBTC	`0xE374DEF41E1c63d54FFeB7B4a3fbA554574A4F84`
